

UNIVERSITÀ DEGLI STUDI DI CATANIA
Anno Accademico 2024 - 2025
Corso di Laurea in Informatica
Prova scritta di Programmazione 1 e laboratorio (A-E / O-Z)
01/09/25

COGNOME e NOME: (IN STAMPATELLO)	
N. MATRICOLA:	

**Non sono consentiti formulari, appunti, libri e calcolatori; non è consentito comunicare con i colleghi; ogni mezzo di comunicazione elettronico deve essere tenuto spento.
Il test va sempre consegnato prima dell'uscita dall'aula.**

Per ciascuna delle seguenti venti domande indicare l'unica risposta corretta.
La prova si considera superata con almeno undici risposte corrette.

1) Il seguente frame di codice

```
#define Z ... // Z e' un qualunque intero maggiore di 1
int alpha = Z;
while(--alpha>0)
    printf("%d", alpha);
```

- ☐ produrrà Z iterazioni;
- ☐ produrrà Z-1 iterazioni;
- ☐ produrrà Z+1 iterazioni;
- ☐ contiene un errore;

2) Completare il seguente frame di codice con l'unica istruzione corretta:

```
struct info{
    char *data[10];
}I1;
// ..
???
```

- ☐ printf("%s", I1.data);
- ☐ printf("%s", *I1.data[rand()%10]);
- ☐ printf("%s", I1.*data[rand()%10]);
- ☐ printf("%s", I1.data[rand()%10]);
- ☐ printf("%s", I1->data[rand()%10]);

3) La seguente istruzione:

```
printf("%zd", sizeof(int));
```

- ☐ il corrispondente output dipenderà dalla piattaforma;
- ☐ il corrispondente output sarà 4;
- ☐ il corrispondente output sarà 8;
- ☐ il corrispondente output sarà 16;
- ☐ nessuna delle risposte precedenti è corretta;

4) Il seguente frame di codice:

```
for(float x = 0.1; x!=1.0; x+=0.1)
    printf("\n%f", x);
```

- ☐ produrrà un numero di iterazioni indefinito;
 - ☐ produrrà 9 iterazioni;
 - ☐ produrrà 10 iterazioni;
 - ☐ produrrà 11 iterazioni;
 - ☐ nessuna delle risposte precedenti è corretta;
-

5) Con riferimento al seguente frame di codice:

```
unsigned ** foo(){
    unsigned *A = malloc(sizeof(unsigned*)*10);
    // ...
    return &A;
}
```

- ☐ essendo il puntatore *A* una variabile locale, l'array sarà deallocato automaticamente con l'uscita dalla funzione stessa;
 - ☐ il puntatore restituito dalla funzione potrà essere usato solo come puntatore `void`;
 - ☐ il codice della funzione contiene almeno un errore;
 - ☐ nessuna delle risposte precedenti è corretta;
-

6) Completare la seguente istruzione in linguaggio C per l'inizializzazione di un array di double con numeri pseudo-casuali

```
unsigned L = 10, i=0;
double A[L+1];
???
```

- ☐ `while(i<=L) A[++i] = rand()*1.0/RAND_MAX;`
 - ☐ `while(i<L+1) A[i++] = rand()/RAND_MAX*1.0;`
 - ☐ `while(i++<L+1) A[i] = rand()*1.0/RAND_MAX;`
 - ☐ `while(i<L+1) A[i++] = rand()*1.0/RAND_MAX;`
-

7) Con riferimento al seguente frame di codice C:

```
char *str = "abcdefg";
str[5] = '*';
```

- ☐ a seguito della sua esecuzione, il sesto carattere della stringa sarà uguale al carattere '*';
 - ☐ a seguito della sua esecuzione, il quinto carattere della stringa sarà uguale al carattere '*';
 - ☐ produrrà warning del compilatore;
 - ☐ produrrà errore a tempo di esecuzione;
 - ☐ nessuna delle precedenti risposte è corretta;
-

8) È possibile allocare un vettore di n elementi di tipo `double*`, in linguaggio C con la seguente istruzione:

- ☐ `double x[] = malloc(sizeof(double*)*n);`
 - ☐ `double* x[] = malloc(sizeof(double*)*n);`
 - ☐ `double* x = malloc(sizeof(double)*n);`
 - ☐ `double* x[n];`
 - ☐ nessuna delle precedenti risposte è corretta;
-

9) con riferimento alla struttura dati CODA implementata mediante lista dinamica:

- ☐ è necessario mantenere un unico puntatore all'elemento in coda;
 - ☐ l'inserimento di un elemento richiede una visita completa della coda a partire dalla testa;
 - ☐ la cancellazione di un elemento richiede il puntatore alla coda;
 - ☐ l'inserimento di un nuovo elemento richiede il puntatore alla testa;
 - ☐ nessuna delle precedenti risposte è corretta;
-

10) Scegliere l'istruzione corretta per la definizione e contestuale inizializzazione di una stringa s :

- ☐ `char *s[] = "my_string";`
 - ☐ `char s[] = {'m', 'y', '_', 's', 't', 'r', 'i', 'n', 'g', 0};`
 - ☐ `char s*[] = "my_string";`
 - ☐ `char *s[] = {'m', 'y', '_', 's', 't', 'r', 'i', 'n', 'g', 0};`
-

11) Completare il corpo della funzione `fill()` mediante inserimento di una singola istruzione:

```
struct record{
    char *str;
    //...
};
void fill(struct record *ptr, const char *source){
    ???
    strcpy( ptr->str , source);
}
```

- ☐ `ptr->str = NULL;`
 - ☐ `ptr->str = char[strlen(source)+1];`
 - ☐ `ptr->str = malloc(sizeof(char)*strlen(source));`
 - ☐ nessuna delle precedenti risposte;
-

12) Completare il seguente frame di codice con una delle quattro proposte elencate in seguito

```
void bar(unsigned K, unsigned L, int ***V){
    for(unsigned i=0; i<K; i++)
        for(unsigned j=0; j<L; j++)
            ??? = rand()%10000;
}
```

- ☐ `** (V[i]+j)`
- ☐ `**V[i+j]`
- ☐ `**V[i]+j`
- ☐ `V[i][j]`

13) Sia `source.o` un file object che contiene l'implementazione di alcune funzioni, ed inoltre sia `main.c` un file sorgente che contenga la funzione `main()`. Indicare il comando apposito che permetta di ottenere un programma eseguibile:

- ☐ `$ gcc -c source.o main.c`
 - ☐ `$ gcc -E source.o main.c`
 - ☐ `$ gcc source.o main.c`
 - ☐ nessuna delle precedenti risposte
-

14) Completare opportunamente il seguente frame di codice selezionando una delle opzioni indicate nel seguito:

```
void foo(char **V, int ns){
    for(int i=0; i<ns; i++){
        char *ptr = V[i];
        ???
    }
}
```

- ☐ `while(++ptr) {printf("%c", *ptr);}`
 - ☐ `while(ptr++) {printf("%c", *ptr);}`
 - ☐ `while(*ptr) {printf("%c", *(ptr+1));}`
 - ☐ `while(*ptr) {printf("%c", *ptr++);}`
-

15) Per il seguente frame di codice:

```
void foo(){
    static double gamma = 0.5;
    gamma*=gamma;
    printf("foo(), gamma=%f", gamma);
}
```

- ☐ il qualificatore `static` permette di re-inizializzare `gamma` ad ogni invocazione della funzione `foo`;
 - ☐ dopo n invocazioni della funzione `foo()`, ultimo valore di `gamma` mostrato sullo standard output sarà uguale a 0.5^n ;
 - ☐ dopo n invocazioni della funzione `foo()`, ultimo valore di `gamma` mostrato sullo standard output sarà uguale a 0.5^{n+1} ;
 - ☐ se una variabile locale è qualificata `static`, questa non potrà essere modificata all'interno della funzione stessa, ma solo inizializzata;
 - ☐ nessuna delle risposte precedenti è corretta;
-

16) Sia `W` il nome di un array monodimensionale precedentemente allocato nel segmento `STACK` (allocazione automatica):

- ☐ `W` sarà deallocato automaticamente con la terminazione del programma;
 - ☐ `W` potrà essere deallocato invocando la funzione `free()`;
 - ☐ `W` sarà deallocato a seguito dell'uscita, del flusso di esecuzione, dalla funzione in cui `W` è stato dichiarato/allocato;
 - ☐ nessuna delle risposte precedenti è corretta.
-

17) Nella codifica di un numero in virgola mobile che si basi sullo standard IEEE 754 a 32 bit (singola precisione):

- ☐ non vi è alcun bit che rappresenta il segno, il quale è rappresentato implicitamente nel campo esponente;
 - ☐ la cosiddetta precisione P dipende dall'ampiezza (numero di bit) del campo esponente;
 - ☐ la precisione P è uguale a 6;
 - ☐ l'intervallo rappresentabile è uguale a $[-2^{32} - 1, 2^{31} - 1]$;
 - ☐ nessuna delle risposte precedenti è corretta;
-

18) In corrispondenza della seguente istruzione:

```
int y = INT_MAX * INT_MAX;
```

- ☐ si avrebbe errore in fase di esecuzione;
 - ☐ si avrebbe errore in fase di compilazione;
 - ☐ si avrebbe warning in fase di compilazione;
 - ☐ nessuna delle precedenti risposta è corretta;
-

19) il comando di shell `$ man printf`

- ☐ chiede alla shell di mostrare un esempio d'uso della funzione `printf()`;
 - ☐ chiede alla shell di compilare le sole translation unit che contengono una chiamata a `printf()`;
 - ☐ chiede alla shell di invocare il programma `man` con argomento `printf`;
 - ☐ nessuna delle precedenti risposta è corretta;
-

20) Completare il seguente frame di codice per il calcolo del montante M di un capitale C dopo X anni ad un tasso T

```
#define C 1000.0
#define T 0.1
#define X 5
```

```
double M = C;
int i = 0;
```

```
while( ??? )
    M+=M*T;
```

- ☐ `i++<=X`
- ☐ `++i<=X`
- ☐ `i++<X`
- ☐ `++i<X`